High Performance Data Processing

Project 1

Optimizing High Performance Data Processing for Large Scale Web Crawlers

Item	Details
Course Title	High Performance Data Processing
Project Title	Optimizing High Performance Data Processing for Large Scale Web Crawlers
Project Type	Group Project (Maximum 3 students per group)
Duration	4 Weeks
Weightage	15% of total course assessment
Submission Date	Monday, 25 May 2025
Submission Channel	e-Learning Portal and GitHub Repository

1. Introduction

Welcome to Project 1 of the High Performance Data Processing course. This project provides a structured opportunity for students to apply core concepts of high performance computing in a realistic data engineering scenario. The activities in this brief are designed to reinforce theoretical knowledge through hands on implementation, performance analysis, and teamwork.

Throughout the project, students will design, build, and optimize a large scale web crawler that collects and processes data from a Malaysian website. The work emphasizes engineering discipline, ethical data collection, and measurable performance improvement using parallel and distributed computing techniques.

This document serves as the official assignment brief. Students are advised to read every section carefully before beginning the project, and to refer back to it throughout the four week implementation period.

2. Project Overview

In modern data driven organizations, the ability to collect, process, and analyze very large datasets is a fundamental skill. This project simulates such an environment by requiring each group to build a complete data pipeline that operates at scale.

Each group will develop a web crawler capable of extracting at least 100,000 structured records from a single Malaysian website. The collected data must then be cleaned, transformed, and stored in an organized manner. Following this, students will apply a range of high performance computing techniques to improve the efficiency of their pipeline. Finally, a structured performance evaluation must be conducted to compare results before and after optimization.

2.1 What Makes This Project High Performance

The project is considered high performance because it goes beyond simple sequential scripting. Students are expected to apply established parallel and distributed computing methods that improve speed, scalability, and resource utilization. The most relevant techniques include the following:

- Multithreading, which allows multiple tasks to run concurrently within a single process.
- Multiprocessing, which uses multiple CPU cores to execute different parts of a workload in parallel.
- Distributed computing frameworks such as Apache Spark or Dask, which spread the workload across multiple processes or machines.

Students must also conduct a clear performance comparison between the unoptimized and optimized versions of their pipeline, supported by quantitative evidence.

3. Learning Outcomes

Upon successful completion of this project, students will be able to:

1. Design and develop a functional web crawler that collects large volumes of real world data from a Malaysian website.
2. Apply data cleaning, transformation, and structured storage techniques on a dataset of at least 100,000 records.
3. Implement at least two high performance computing techniques to improve the efficiency of a data processing pipeline.
4. Evaluate system performance using objective metrics such as execution time, memory usage, CPU utilization, and throughput.
5. Collaborate effectively in a small team and contribute meaningfully to a shared technical deliverable.
6. Communicate technical results professionally through a written report and an oral presentation.

4. Project Requirements

4.1 Group Composition

This is a group based project. Students must form their own groups subject to the following requirements:

- **Each group must consist of a maximum of three (3) students.** Groups with more than three members will not be accepted.
- Each group must include members from different genders, races, or backgrounds in order to encourage diverse perspectives and inclusive collaboration.
- Roles and responsibilities must be distributed clearly among members. Suggested roles include Group Leader, Data Engineer, HPC Specialist, and Documentation Lead.
- Group formation must be confirmed with the lecturer no later than the end of Week 1.

4.2 Technical Requirements

The technical scope of the project is fixed. Each group must satisfy all of the following criteria:

- Crawl and collect at least 100,000 valid, structured records from one approved Malaysian website.
- Store the collected data in a structured format such as CSV, JSON, or a relational database.
- Apply data cleaning operations such as removing duplicates, standardizing fields, handling missing values, and validating data types.
- Implement at least two distinct optimization techniques drawn from threading, asyncio, multiprocessing, Dask, or Spark.
- Conduct a performance comparison between the baseline and optimized versions, supported by charts or tables.

- Respect the target website by honoring robots.txt rules, applying appropriate crawl delays, and avoiding any behavior that may overload the server.

4.3 Suggested Malaysian Websites

Each group must select only one website from the list below. Approval from the lecturer is required before beginning data collection.

No.	Website	Domain	Example Data Fields
1	JobStreet (jobstreet.com.my)	Job Search	Job title, company name, location, salary, job category
2	Lazada (lazada.com.my)	E commerce	Product name, price, category, seller, ratings
3	Mudah.my	Online Marketplace	Listing title, item type, price, region, seller
4	PropertyGuru (propertyguru.com.my)	Real Estate	Property title, price, location, property type
5	NST (nst.com.my)	News	Headline, publication date, section, summary
6	SoyaCincau (soyacincau.com)	Tech Reviews	Article title, brand, topic, publication date
7	CARSOME (carsome.my)	Car Sales	Car model, year, price, mileage, dealer name
8	MyFutureJobs (myfuturejobs.gov.my)	Government Job Portal	Job title, employer, salary, sector, region
9	iProperty (iproperty.com.my)	Real Estate	Listing name, price, number of rooms, location

Important note: Students must verify that the chosen website can realistically supply the required volume of records before requesting approval.

5. Technical Tasks

The project is organized into a four week schedule. Each week has clear objectives and milestones. Groups are encouraged to follow this schedule closely to ensure steady progress.

5.1 Weekly Schedule

Week	Phase	Key Activities
Week 1	Planning and Setup	Form the group, select a Malaysian website, obtain lecturer approval, identify target data fields, and design the overall system architecture.
Week 2	Crawler Development	Implement the baseline web crawler. Apply ethical scraping practices, save data progressively, and begin populating the dataset.
Week 3	Processing and Optimization	Clean and transform the dataset, apply at least two optimization techniques, and benchmark the system before and after optimization.
Week 4	Reporting and Submission	Compile final results, write the report, prepare presentation slides, and submit all deliverables through the required channels.

5.2 Suggested Role Assignments

Although roles may be adapted to suit each group, the following structure is recommended for a team of three:

Role	Primary Responsibilities
Group Leader and Data Engineer	Coordinates the team, oversees the crawler, manages data ingestion and storage, and ensures progress against the timeline.
HPC and Performance Specialist	Designs and implements optimization techniques, runs benchmarks, and produces performance comparison charts.
Analyst and Documentation Lead	Performs data cleaning and analysis, prepares the final report, and develops the presentation slides.

Note: Every member is expected to contribute to coding, testing, and documentation regardless of their primary role.

5.3 Performance Evaluation Requirements

To demonstrate the impact of optimization, each group must measure and report the following metrics for both the baseline and optimized versions of the pipeline:

- Total processing time, expressed in seconds or minutes.
- Average CPU utilization during execution.
- Peak memory consumption.
- Throughput, measured as the number of records processed per second.

Results must be presented using clearly labeled tables and charts. A short written interpretation of the findings is expected.

6. Deliverables

Each group must submit the following five deliverables. All items will be assessed as part of the final grade.

No.	Deliverable	Description
1	Final Report (PDF)	A complete technical report covering background, methodology, implementation, results, and discussion. Must be uploaded to Turnitin in PDF format.
2	Source Code	The complete crawler and processing pipeline, organized clearly and accompanied by inline comments. Submit as a GitHub repository link or a ZIP archive.
3	Cleaned Dataset	At least 100,000 valid and structured records in CSV, JSON, or database format.
4	Performance Comparison	Tables and charts presenting before and after optimization metrics, including time, CPU, memory, and throughput.
5	Presentation Slides	Slide deck for a 10 minute group presentation summarizing the project.

6.1 Recommended GitHub Folder Structure

To maintain consistency and ease of grading, please follow the recommended folder structure when submitting your repository:

```
📁 p1/[your group]
│
├── 📁 data/
│   └── raw_data.json
│   └── cleaned_data.csv
├── 📁 p1/
│   └── main_crawler.ipynb
│   └── clean_data.ipynb
│   └── optimize_pipeline.ipynb
│
├── 📁 p2/
```

```
│   └── performance_before.csv
│   └── performance_after.csv
│   └── evaluation_charts.ipynb
│
├── 📁 report/
│   └── Final_Report.pdf
│   └── Presentation_Slides.pptx
│
├── README.md
└── requirements.txt
```

Note: The folder structure shown above is presented for reference. In your actual repository, the line breaks should appear as separate folders and files.

7. Submission Guidelines

All submissions must follow the rules below. Late or incomplete submissions may result in a deduction of marks at the lecturer's discretion.

7.1 Submission Channels

Channel	Purpose
e-Learning Portal	Upload the final report (PDF), presentation slides (PPTX), and a link to your GitHub repository.
Turnitin	Upload the final report (PDF) for plagiarism verification.
GitHub Repository	Host all source code, datasets, notebooks, and supporting files using the recommended folder structure.

7.2 Submission Deadline

All deliverables must be submitted by the following deadline:

Monday, 25 May 2025, before 11:59 pm (Malaysian Standard Time).

7.3 Suggested Final Report Structure

The final report should follow a professional structure. The recommended sections are listed below:

1. Cover Page, including project title, course name, group members, matric numbers, and submission date.
2. Table of Contents.
3. Introduction, covering background, objectives, target website, and the data to be extracted.
4. System Design and Architecture, including a diagram, tools and frameworks used, and team roles.
5. Data Collection, describing the crawling method, number of records collected, and ethical considerations.
6. Data Processing, including cleaning methods, data structure, and transformation steps.
7. Optimization Techniques, with a description of each method applied and supporting code excerpts.
8. Performance Evaluation, presenting before and after results with appropriate charts.
9. Challenges and Limitations encountered during the project.
10. Conclusion and Future Work.
11. References.
12. Appendices, containing sample code, screenshots, and links to the repository or dataset.

8. Evaluation Criteria

The project will be assessed using the rubric shown below. The total weightage is 100 percent of the project mark, which contributes 15 percent to the overall course grade.

Assessment Area	Description	Weight
Problem Understanding	Clear objectives, dataset identification, and ethical crawling considerations.	10%
System Design	Architecture planning, tool selection, and modular system design.	15%
Crawler and Processing Implementation	Functional pipeline that successfully collects at least 100,000 records.	25%
Optimization Techniques	Correct and effective use of HPC methods.	20%
Performance Evaluation	Quality of metric comparison, accuracy of results, and chart presentation.	10%
Final Report	Clarity, completeness, and overall professionalism of the document.	10%
Group Presentation	Teamwork, clarity of explanation, time management, and Q and A handling.	10%
Total	**Sum of all assessment areas.**	**100%**

9. How to Score High Marks

To achieve a strong grade in this project, groups should focus on the following best practices:

- Choose a website with a realistic and reliable supply of data. Confirm the data volume before committing to the topic.
- Document every design decision in the report. Explain why a particular library, framework, or optimization technique was selected.
- Apply at least two genuinely different optimization techniques. Combining threading with multiprocessing or Spark generally produces stronger results than two similar approaches.
- Present performance results using both numerical tables and visual charts. Discuss the meaning of the numbers, not only the numbers themselves.
- Maintain a clean GitHub repository. Use clear file names, write a complete README, and ensure the requirements.txt file is up to date.
- Practice the presentation in advance. Each member should be able to explain any part of the system.

10. Recommended Tools and Techniques

The following tools and libraries are widely used and well documented. Groups are not limited to this list, but it provides a strong starting point.

Category	Recommended Tools
Web Crawling	Requests, BeautifulSoup, Scrapy, Selenium, Playwright.
Asynchronous Crawling	asyncio, aiohttp, httpx.
Parallel Processing	Python threading, multiprocessing, concurrent.futures, joblib.
Distributed Computing	Apache Spark (PySpark), Dask, Modin.
High Performance Data Frames	Polars, DuckDB, Pandas with vectorization.
Data Storage	CSV, JSON, SQLite, PostgreSQL, MongoDB.
Visualization and Reporting	Matplotlib, Seaborn, Plotly, Jupyter Notebook.

Useful reference repositories that may support your implementation:

- Web Scraping examples: github.com/drshahizan/python-web
- Big Data Processing examples: github.com/drshahizan/Python-big-data

11. Example Workflow

The following workflow illustrates a typical end to end project execution. Groups may adapt these steps to suit their chosen website and tools.

1. Identify the data source. Inspect the structure of the target website and confirm that at least 100,000 records can be obtained.
2. Plan the data fields. Decide which attributes will be extracted and how they will be stored.
3. Build a baseline crawler. Implement a single threaded version that collects a small sample of records.
4. Scale the crawler. Extend the baseline to handle pagination, rate limiting, error recovery, and progressive saving.
5. Clean the dataset. Remove duplicates, handle missing values, standardize formats, and validate data types.
6. Benchmark the baseline. Record execution time, CPU usage, memory consumption, and throughput.
7. Apply optimization. Introduce at least two HPC techniques such as multithreading combined with PySpark.
8. Benchmark the optimized version. Use the same dataset and metrics to ensure a fair comparison.
9. Visualize and analyze. Produce charts and discuss the improvement in clear, accessible language.
10. Document and present. Compile the report, polish the slides, and rehearse as a team.

12. Common Mistakes to Avoid

Many groups encounter the same challenges. Being aware of them in advance can save valuable time.

- Choosing a website that does not actually contain 100,000 records. Always verify before committing.
- Ignoring robots.txt or sending requests too aggressively. This may result in IP bans or ethical concerns.
- Saving data only at the end of the crawl. Always save progressively to avoid losing hours of work.
- Selecting two optimization techniques that are too similar to provide a meaningful comparison.
- Reporting performance numbers without explaining their meaning or implications.
- Submitting code without comments, documentation, or a clear folder structure.
- Leaving the report writing to the last day. Build it gradually as the project progresses.

13. Tips for Success

- Start early. The 100,000 record threshold often requires more time than expected.

- Use version control properly. Commit frequently and write meaningful commit messages.
- Hold short weekly meetings to track progress and resolve blockers.
- Ask the lecturer for guidance whenever you are unsure about scope or approach.
- Test on a small dataset first, then scale up gradually to avoid wasting compute resources.
- Always keep a backup copy of the dataset and code outside the main GitHub repository.

14. Important Notes

Please pay close attention to the following points:

- Plagiarism, including copying code or text without proper attribution, will be treated as a serious academic offense.
- Each group member must contribute fairly. Imbalanced contributions may affect individual marks.
- All datasets must be obtained legally and ethically. Always honor the terms of service of the target website.
- Submitting outside the official channels listed in this brief will result in the submission being treated as missing.
- Any questions regarding scope or grading should be directed to the lecturer or teaching assistant during the course period.

End of Assignment Brief